VPS Server Specification & Architecture

Complete technical blueprint, system topology, security policies, and standard operating procedures.

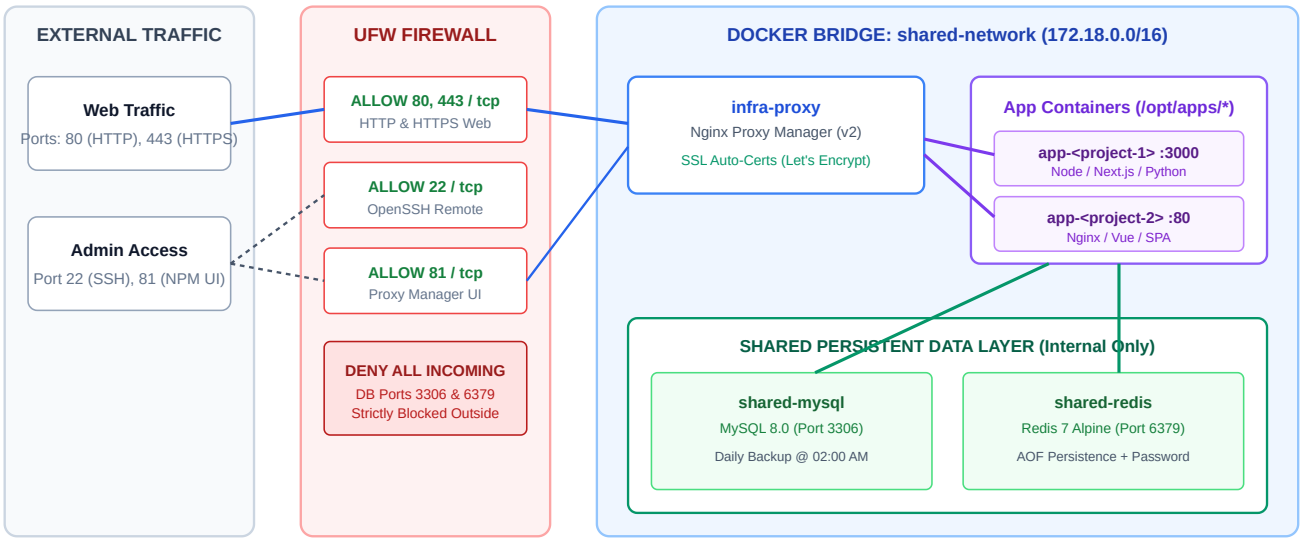
IPv4: 172.255.210.203 | SSH: vps-prod | OS: Ubuntu 24.04.5 LTS | CPU: 6 vCPU AMD EPYC | RAM: 16 GB (15 GiB Usable) | Storage: 200 GB SSD

1 Live Hardware & Operating System Specifications

> Hardware specs verified live from the KVM hypervisor and Linux kernel `6.8.0-146-generic` via `lscpu`, `free -h`, `lsblk`, and `df -h`.

SUBSYSTEM	CONFIGURATION	LIVE OPERATIONAL DETAILS
Virtualization	CloudStack KVM Hypervisor	Full KVM Hardware Virtualization (QEMU/KVM `x86_64`)
Operating System	Ubuntu 24.04.5 LTS	Noble Numbat (Kernel `6.8.0-146-generic` SMP PREEMPT_DYNAMIC)
Processor (CPU)	6 vCPU AMD EPYC	6 Cores / 1 Socket / 6 Threads @ 2.0 GHz (BogoMIPS: 5589.49)
CPU Cache	Multi-Tier Dedicated	L1d: 192 KiB, L1i: 384 KiB, L2: 3 MiB, L3: 8 MiB (Single Instance)
Memory (RAM)	16 GB (15 GiB Usable)	16,376,316 kB Total | ~1.0 GiB In Use | ~14.6 GiB Available
Swap Space	4.0 GB Active	Dedicated `/swapfile` (Priority -2)
Primary Disk Storage	200 GB SSD (ext4)	`/dev/sda2`: 197 GB Total | 9.8 GB Used (6%) | 179 GB Available
Network Interface	`eth0` (Gigabit Ethernet)	IPv4: `172.255.210.203/24` | IPv6: `2402:a7c0:3003:115::d6b/64`
Docker Engine	Version 29.9.2 (build a82e528)	Storage Driver: `overlay2` | Logging: `json-file` (10m max, 3 files)
Docker Compose	Version v5.7.0	Integrated Native Docker Compose v2 CLI plugin

2 Infrastructure Architecture & Network Topology



3 Standard Directory Standard (`/opt`)

All core infrastructure, container configurations, persistent storage volumes, and automation scripts reside in `/opt`:

```
/opt/
├── infra/
│   ├── proxy/                    # Nginx Proxy Manager Stack
│   │   ├── docker-compose.yml    # Reverse proxy & SSL manager service definition
│   │   ├── data/                 # NPM database, access lists & configs
│   │   └── letsencrypt/          # Automated Let's Encrypt certificates
│   └── database/                 # Shared MySQL & Redis Stack
│       ├── .env                  # Root passwords (mode 0600 - root only)
│       ├── docker-compose.yml    # MySQL 8.0 & Redis 7 services
│       ├── data/                 # Persistent MySQL data (/var/lib/mysql, mode 0700)
│       └── init/                 # Custom init SQL scripts (/docker-entrypoint-initdb.d)
├── apps/                         # Application Deployments
│   └── <project-slug>/           # Isolated stack per project
│       ├── docker-compose.yml    # Service definition (attaches to shared-network)
│       ├── .env.production       # Project-specific environment variables
│       └── app/                  # Codebase repository & Dockerfile
└── ops/                          # Operations, Automation & Maintenance
    ├── backups/                  # Automated Database Backups (mode 0700)
    │   ├── db-backup.sh          # Daily automated mysqldump script
    │   └── archives/             # Rolling 7-day compressed archives (.sql.gz)
    └── maintenance/
        └── docker-cleanup.sh     # Weekly Docker builder & image pruning script
```

4 Running Containers & Network Isolation

CONTAINER NAME	DOCKER IMAGE	INTERNAL PORTS	PUBLIC HOST PORTS	RESTART POLICY	ATTACHED VOLUMES
`infra-proxy`	`jc21/nginx-proxy-manager:latest`	`80`, `443`, `81`	`0.0.0.0:80, 443, 81`	`unless-stopped`	`./data`, `./letsencrypt`
`shared-mysql`	`mysql:8.0`	`3306`, `33060`	**None (Blocked)**	`unless-stopped`	`./data` (`/var/lib/mysql`), `./init`
`shared-redis`	`redis:7-alpine`	`6379`	**None (Blocked)**	`unless-stopped`	In-memory with AOF persistence

5 Security Protocols & Firewall Hardening

> **Strict Isolation Policy:** Application stacks must never bind database ports (3306, 6379) to host interfaces (0.0.0.0). All container traffic must communicate strictly over the internal bridge `shared-network`.

SECURITY DOMAIN	CONFIGURATION & POLICY	IMPLEMENTATION DETAILS
UFW Firewall	Default Deny Incoming | Allow Outgoing	Whitelisted: `22/tcp` (SSH), `80/tcp` (HTTP), `443/tcp` (HTTPS), `81/tcp` (NPM UI)
SSH Authentication	Ed25519 Key-Based Authentication	Configured in `~/.ssh/authorized_keys`; accessible locally via alias `ssh vps-prod`
Root Credentials	Restricted Read-Only Access (`chmod 600`)	Stored exclusively at `/opt/infra/database/.env`; apps connect via dedicated accounts
Proxy Management	Nginx Proxy Manager Admin Console	Accessible via `http://172.255.210.203:81` (Auto-provisions Let's Encrypt SSL)

6 Automated Backups & System Maintenance

TASK	SCHEDULE (CRON)	SCRIPT TARGET	RETENTION POLICY
MySQL Full Dump	`0 2 * * *` (Daily 02:00 UTC)	`/opt/ops/backups/db-backup.sh`	7-day rolling `.sql.gz` archives
Docker Garbage Prune	`0 4 * * 0` (Weekly Sun 04:00 UTC)	`/opt/ops/maintenance/docker-cleanup.sh`	Prunes images & builder cache >168h

7 Standard Operating Procedure (SOP): Project Deployment

Step 1: Provision Isolated Database & User

Never connect apps with MySQL `root`. Provision an isolated schema and user:

```
# Local Helper Script
./scripts/provision-db.sh <project-slug>

# Remote Non-Interactive Execution
ssh vps-prod 'bash -s' << 'ENDSSH'
source /opt/infra/database/.env
APP_DB="db_<project-slug>"
APP_USER="user_<project-slug>"
APP_PASS=$(openssl rand -base64 18 | tr -dc 'a-zA-Z0-9' | head -c 16)

docker exec -i shared-mysql mysql -u root -p"$MYSQL_ROOT_PASSWORD" <<SQL
CREATE DATABASE IF NOT EXISTS \`${APP_DB}\` CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER IF NOT EXISTS '${APP_USER}'@'%' IDENTIFIED BY '${APP_PASS}';
GRANT ALL PRIVILEGES ON \`${APP_DB}\`.* TO '${APP_USER}'@'%';
FLUSH PRIVILEGES;
SQL

echo "DB_HOST=shared-mysql"
echo "DB_PORT=3306"
echo "DB_DATABASE=${APP_DB}"
echo "DB_USERNAME=${APP_USER}"
echo "DB_PASSWORD=${APP_PASS}"
ENDSSH
```

Step 2: Configure Application Container (`docker-compose.yml`)

Place in `/opt/apps/<project-slug>/docker-compose.yml` without public host port mappings:

```
services:
  app:
    build:
      context: ./app
      dockerfile: Dockerfile
    container_name: app-<project-slug>
    restart: unless-stopped
    env_file: .env.production
    networks:
      - shared-network
    # IMPORTANT: DO NOT expose host ports (e.g. no "3000:3000" or "80:80")

networks:
  shared-network:
    external: true
```

Step 3: Route Inbound Traffic via Nginx Proxy Manager

1. Log in to **NPM Console** at `http://172.255.210.203:81`.
2. Navigate to **Hosts** → **Proxy Hosts** → **Add Proxy Host**.
3. Set **Domain Names**: e.g. `api.yourdomain.com` or `app.yourdomain.com`.
4. Set **Forward Hostname / IP**: `app-<project-slug>` (resolved automatically via Docker DNS).
5. Set **Forward Port**: `3000` (Node/Next.js) or `80` (Nginx/SPA).

6. Under **SSL Tab**: Select *Request a new SSL Certificate*, enable *Force SSL* and *Block Common Exploits*.